MODULE 3 · MACHINE LEARNING · DAY 37

What Right Means

Evaluation metrics for classification

Yesterday you learned to measure honestly.
Today: measure what.

5	17	1
sections	worked pictures	confusion matrix

Srinivasa Sai Chava · Boston University

WHERE WE ARE

Yesterday ended on a model nobody should ship

The HR model from Day 36's practice

90.6% accurate. Exactly its own do-nothing baseline. It flagged 4 people out of 2,000 and caught 2 of the 187 who actually left.

You found three faults in how it was measured. But the deepest problem was not the measurement — it was that accuracy was the wrong question from the start.

Today's five sections

1. Four numbers, not one
2. Precision and recall
3. Back down to one number
4. The whole curve
5. Putting a price on it

Still no new model.

Day 36 was about whether your number is honest. Today is about whether it is the right number to be asking for.

These two days together are what separates somebody who can fit a model from somebody you would let deploy one.

Accuracy answers a question almost nobody is asking.

"How often is the model right?" sounds like the obvious question. It is the obvious question only when both kinds of mistake cost the same and both answers are equally common — which, in the problems worth solving, is almost never true.

SECTION 1 OF 5

Four numbers, not one

The confusion matrix, and what accuracy throws away

- 6,000 card payments
- 95.8% accurate
- 223 frauds let through

THE PROBLEM

Today's data: 6,000 card payments

```
amount       how much the payment was for
hour         0 to 23
night        1 if between 1am and 5am
recent_txns  payments on this card in the last hour
foreign      1 if the merchant is overseas
channel      chip  /  online  /  contactless

fraud        1 if the payment was fraudulent
```

290

of them fraud

4.83%

of all payments

0.9517

accuracy of 'never block'

What the data already says

online payments are fraudulent 9.8% of the time; chip payments 1.0%

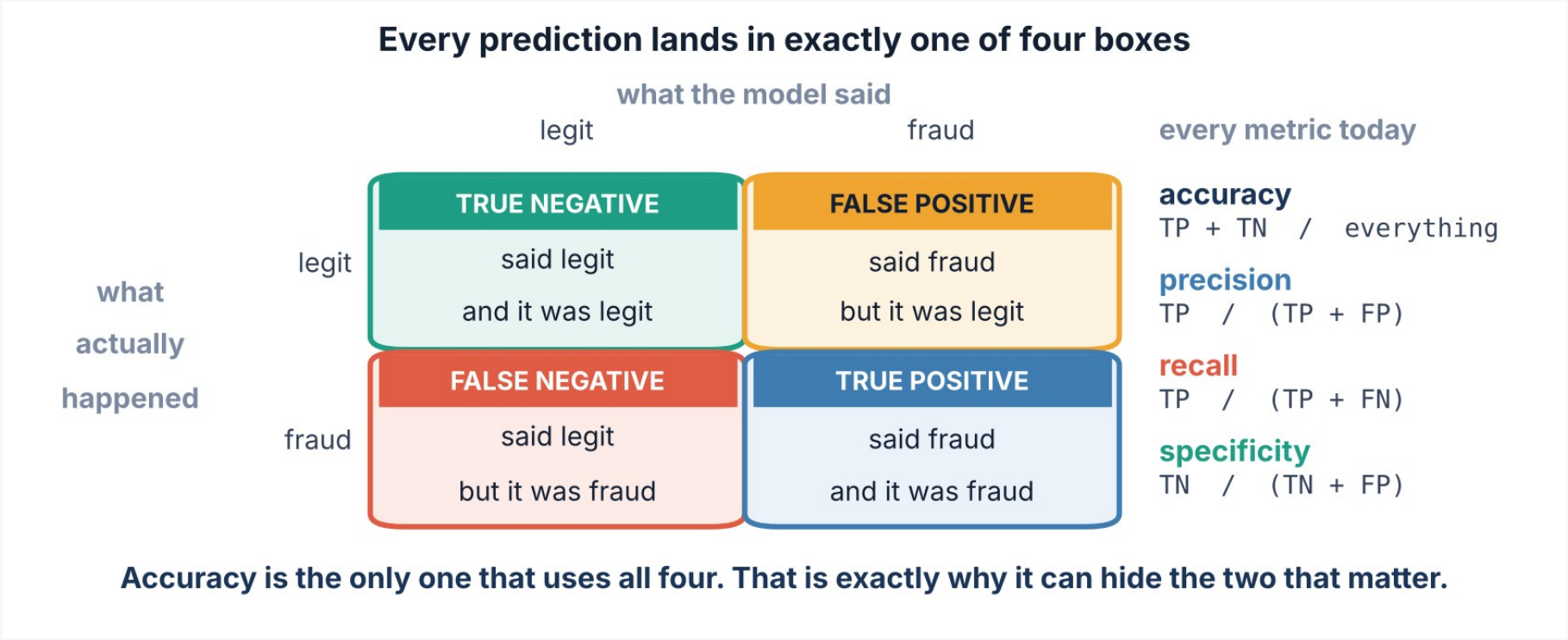
foreign merchants: 21.0% against 3.2%

the median fraud is ₹90.61; the median honest payment ₹35.34

Every number today comes from cross-validated predictions on all 6,000 rows — yesterday's lesson, applied without comment from here on.

THE CONFUSION MATRIX

Every prediction lands in one of four boxes

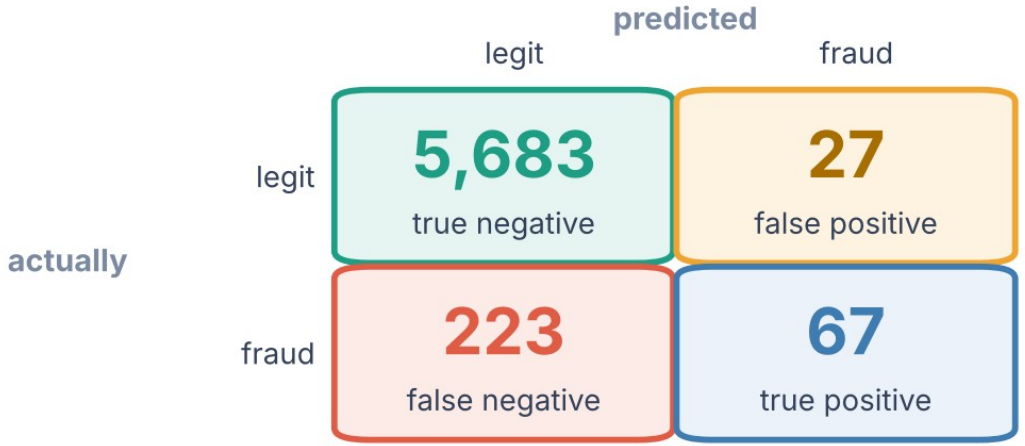


Learn these four names properly today. Every metric for the rest of the course is built out of them, and most arguments about models are really arguments about which box matters.

MEASURED

The same four boxes, with our numbers in them

6,000 card transactions · 290 of them fraud · the default 0.5 cut-off



accuracy 0.9583
and 'always legit' scores 0.9517

precision 0.7128
when it shouts, it is right 67/94

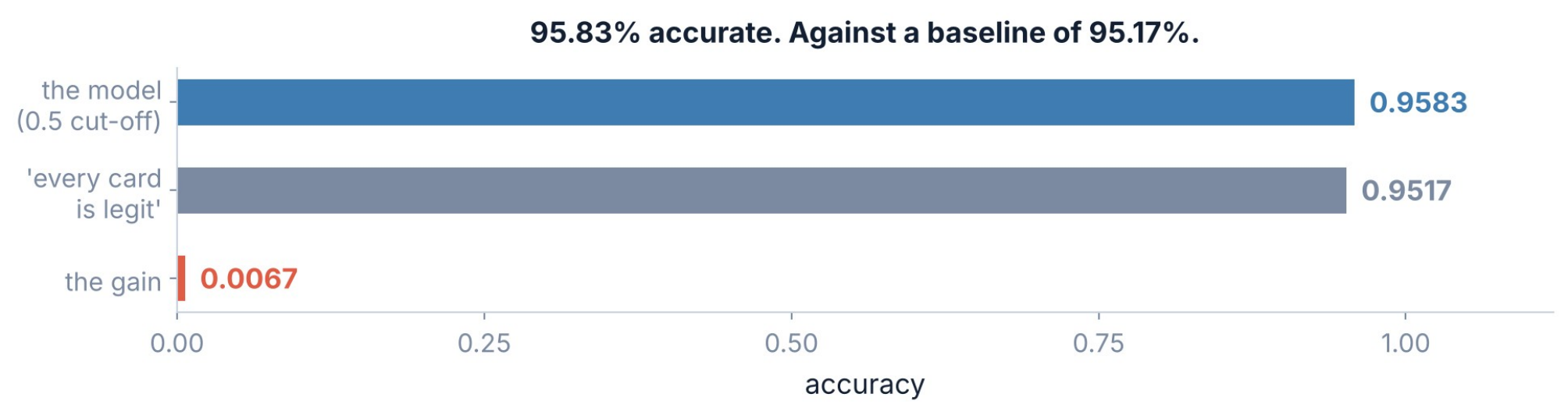
recall 0.2310
it catches 67 of the 290 frauds

223 frauds went through. The accuracy figure does not mention them.

Read the bottom-left box. 223 frauds went through, and not one of the headline numbers you would normally quote mentions them.

ACCURACY AGAINST ITS BASELINE

95.83% accurate. And almost worthless.



The 'always legit' model needs no data, no columns and no training. It is right 95.17% of the time because 95.17% of payments are legitimate.

Two thirds of a percentage point. That is the entire contribution of the model, measured the way most people measure.

SKLEARN.METRICS

The four numbers, in code

```
from sklearn.metrics import confusion_matrix, classification_report

tn, fp, fn, tp = confusion_matrix(y, pred).ravel()    # <- note the order

accuracy    = (tp + tn) / (tp + tn + fp + fn)     # 0.9583
precision   =  tp / (tp + fp)                     # 0.7128
recall      =  tp / (tp + fn)                     # 0.2310
specificity =  tn / (tn + fp)                     # 0.9953
```

Watch the unpacking order

The .ravel() call gives tn, fp, fn, tp — reading the matrix left to right, top to bottom. Get this backwards and precision and recall swap places silently.

SPECIFICITY — THE ONE NOBODY QUOTES

0.9953 of honest payments got through untouched. On imbalanced data this is almost always near 1 and almost always useless — it is dominated by the enormous tn box.

classification_report(y, pred) prints most of this in one line. Use it, but know where each number came from.

SECTION 2 OF 5

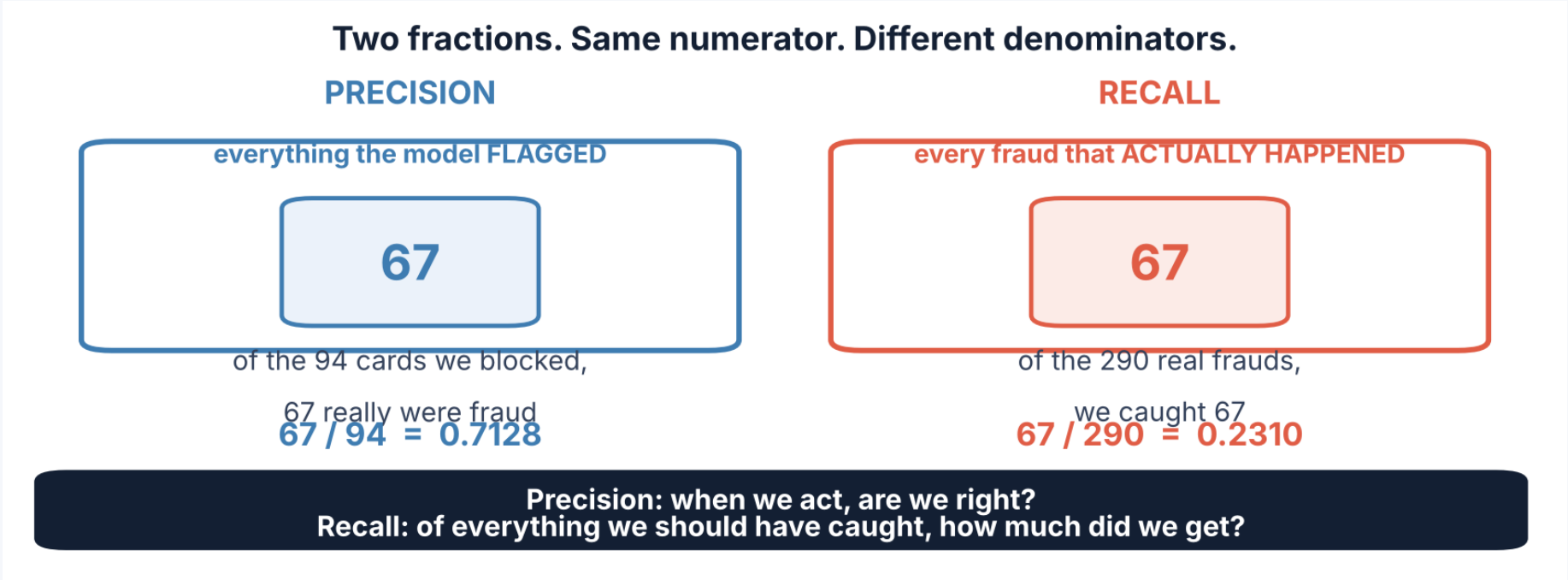
Precision and recall

Two fractions with the same numerator and different denominators

- when we act, are we right?
- how much did we catch?
- and why you cannot have both

SAME TOP, DIFFERENT BOTTOM

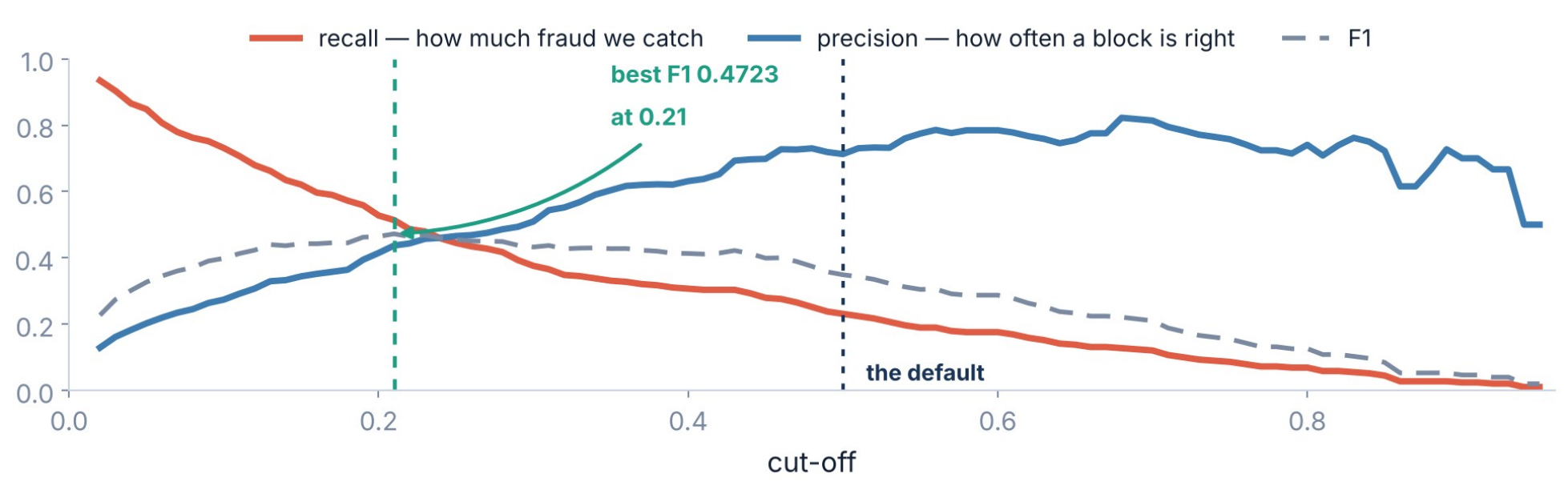
The only two metrics you must never confuse



A way to keep them straight: precision is about the model's claims, recall is about the world's cases. Precision asks about the 94; recall asks about the 290.

THE TRADE-OFF

Move the cut-off and they move in opposite directions



This is not a flaw to be engineered away. Blocking more cards must catch more fraud and must be wrong more often. The curve is the shape of the choice itself.

SAME MODEL THROUGHOUT

Four cut-offs. Four completely different banks.

CUT-OFF 0.05	CUT-OFF 0.2	CUT-OFF 0.5	CUT-OFF 0.9
blocks 1,216 cards	blocks 369 cards	blocks 94 cards	blocks 10 cards
catches 246 of 290	catches 153 of 290	catches 67 of 290	catches 7 of 290
precision 0.202	precision 0.415	precision 0.713	precision 0.700
recall 0.848	recall 0.528	recall 0.231	recall 0.024

Nothing about the model changed between these four. Not a coefficient, not a row of data. Only the number you compare the probability against.

At 0.90 precision actually FALLS, to 0.700. With only 10 cards flagged, one mistake moves it ten points — precision is noisy when the flagged set is tiny.

PICK A ROW

The whole sweep, as a table

cut-off	cards blocked	fraud caught	fraud missed	precision	recall	accuracy
0.05	1,216	246	44	0.2023	0.8483	0.8310
0.10	774	212	78	0.2739	0.7310	0.8933
0.15	523	180	110	0.3442	0.6207	0.9245
0.20	369	153	137	0.4146	0.5276	0.9412
0.30	214	109	181	0.5093	0.3759	0.9523
0.50	94	67	223	0.7128	0.2310	0.9583
0.70	43	35	255	0.8140	0.1207	0.9562
0.90	10	7	283	0.7000	0.0241	0.9523

Accuracy is highest at 0.50 — the row that misses 223 of the 290 frauds. Maximising accuracy on imbalanced data is a way of asking the model to do nothing.

IT DEPENDS ON THE MISTAKE

So which one do you actually want?

RECALL MATTERS MORE

a cancer screen · fraud on a large account · a safety alert on a machine

Missing a case is catastrophic and a false alarm costs a review. Accept the false alarms.

PRECISION MATTERS MORE

blocking a card · auto-rejecting a job application · flagging a customer to the police

Acting wrongly does real harm to a real person. Be sure before you act.

Card fraud is genuinely both, which is why it is a good example.

Miss a fraud and the bank refunds it. Block a real customer and they are stranded at a till. Section 5 settles it with money, because nothing else can.

You cannot answer 'which metric' by looking at the model. You answer it by asking who gets hurt by each kind of mistake.

SECTION 3 OF 5

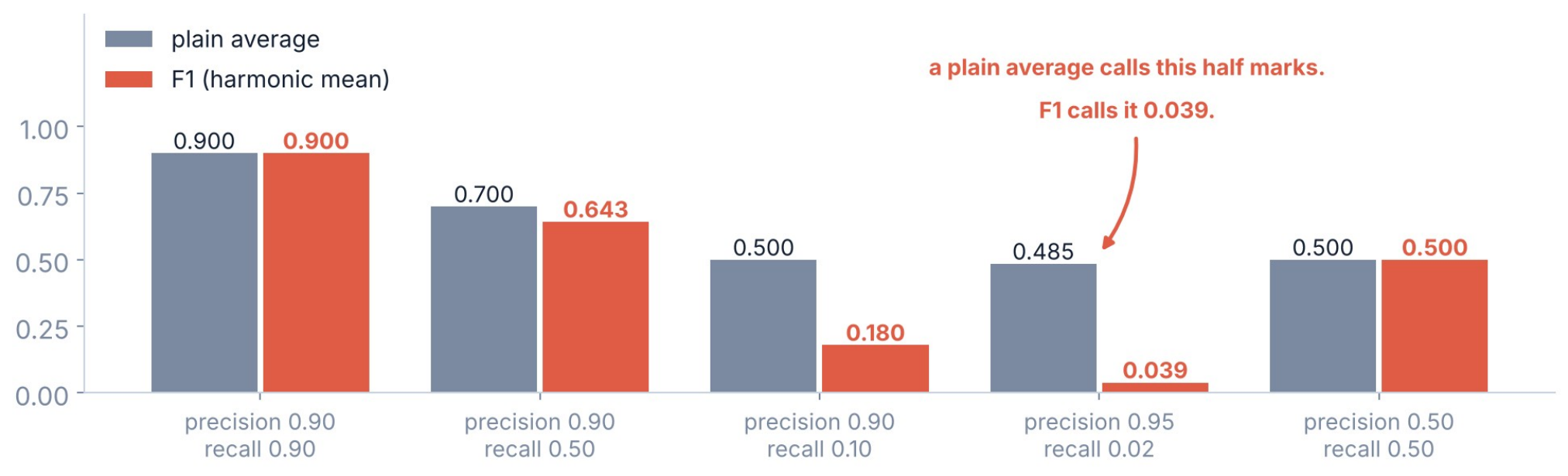
Back down to one number

When you really do have to report a single figure

- F1 and why it is harmonic
- balanced accuracy
- three rules, three answers

NOT A PLAIN AVERAGE

F1 is a harmonic mean, and that is the whole point



Precision 0.95 with recall 0.02 is a model that blocks four cards a year and is usually right. A plain average calls that 0.485. F1 calls it 0.039.

F1 AND F-BETA

The formula, and the one knob on it

$$F1 = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} = 0.3490 \text{ at our 0.5 cut-off}$$

```
from sklearn.metrics import f1_score, fbeta_score
f1_score(y, pred)                  # 0.3490
```

F0.5	**precision counts double**	use when a false alarm is the expensive mistake
F1	**they count equally**	the default, and the one people mean by 'F score'
F2	**recall counts double**	use when a miss is the expensive mistake

beta is how many times more you care about recall than precision. Choosing it is the same decision as choosing a cut-off, moved one step earlier.

BALANCED ACCURACY AND MCC

Two more one-number summaries

Balanced accuracy

The average of recall on each class — here (0.2310 + 0.9953) / 2 = 0.6132.

It asks: how would you score if the two classes were equally common? A model that always says 'legit' scores exactly 0.5000, however rare the fraud is. That is its whole value.

Matthews correlation (MCC)

The only common single number that uses all four boxes and is not fooled by imbalance. Runs from −1 to +1, where 0 is chance.

Statisticians like it. Managers do not understand it. Report it alongside something plainer, never instead.

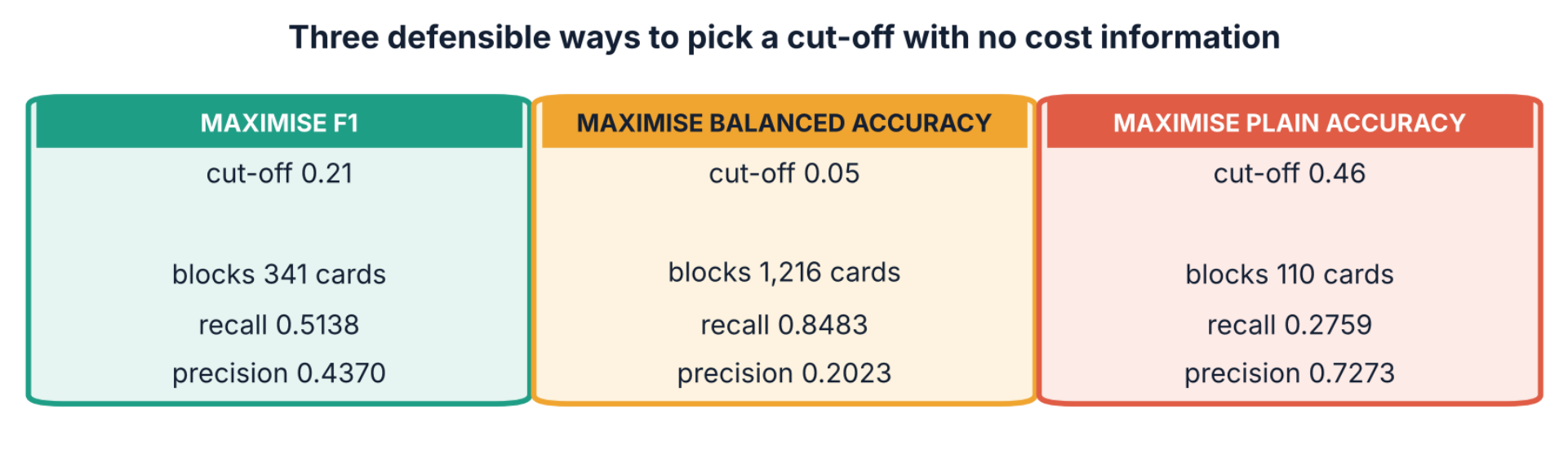
The honest warning about every number on this slide

Each one squeezes a two-dimensional trade-off into one dimension, and to do that it has to assume something about what you care about. F1 assumes precision and recall matter equally. Balanced accuracy assumes the classes do. Neither assumption is likely to be true for your problem.

Use them to compare models on a level field. Do not use them to choose a cut-off — section 5 has a better way.

AND NONE OF THEM IS RIGHT

Three sensible rules. Three different banks.



Each rule sounds defensible in a meeting. They block 341, 1,216 and 110 cards. The disagreement is not a technical detail — it is the entire business decision.

SECTION 4 OF 5

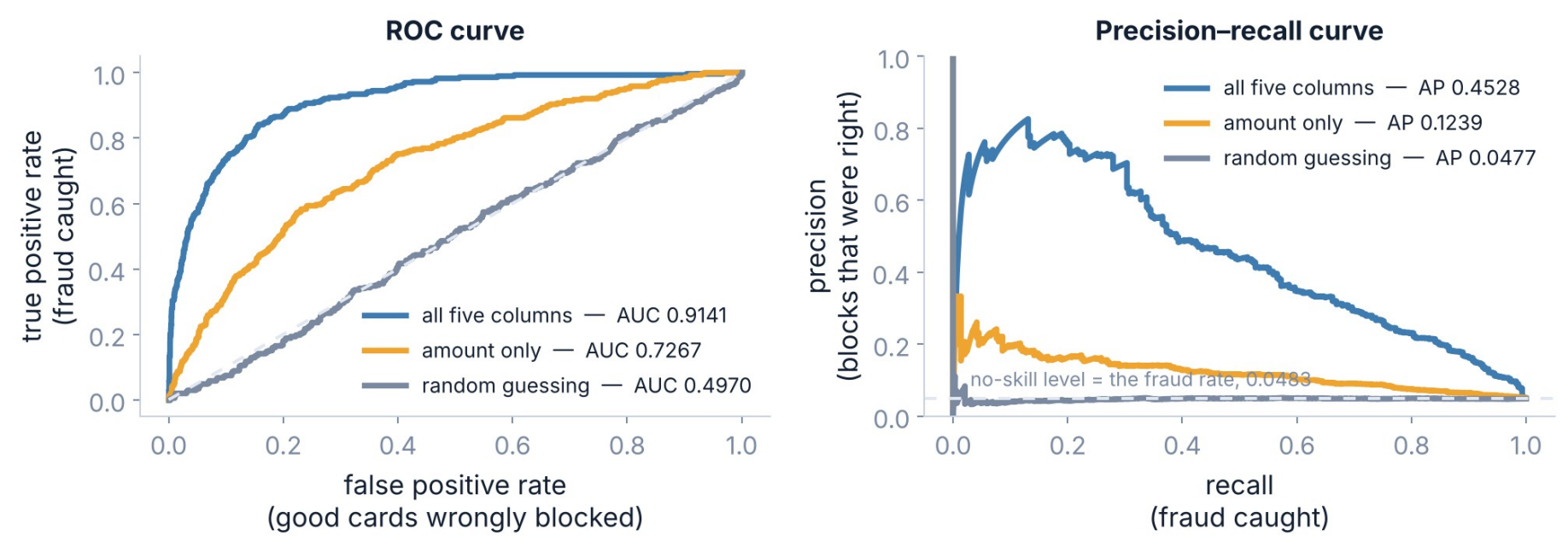
The whole curve, not one point

Scoring a model across every cut-off at once

- ROC and AUC
- what AUC really means
- and when it lies to you

ROC AND PRECISION-RECALL

Two curves, three models



Each point on each curve is one cut-off. The curves say nothing about which cut-off to pick — they summarise how good the ordering is, whatever you pick.

AND HOW TO CHECK IT

What the AUC number actually means

AUC has a plain-English meaning, and it can be checked by hand

THE DEFINITION	CHECKED ON 400,000 PAIRS
Pick one fraud and one legit card at random. AUC is the chance the model gave the fraud the higher score.	The fraud scored higher in 0.9142 of them. roc_auc_score says 0.9141.

So AUC measures the RANKING, not the decisions. It never looks at your cut-off.

Which is why a model can have an excellent AUC and still be useless as deployed — ours catches 23% of fraud at 0.5.

0.5 means the ranking is a coin flip. 1.0 means every fraud outranks every honest payment. Our 0.9141 means it gets the pair right nine times in ten.

TWO IMPORTS

Drawing them, in code

```
from sklearn.metrics import (roc_curve, roc_auc_score,
                             precision_recall_curve, average_precision_score)

probs = model.predict_proba(X)[:, 1]        # NOT .predict()

fpr, tpr, cuts = roc_curve(y, probs)
roc_auc_score(y, probs)                     # 0.9141

prec, rec, cuts = precision_recall_curve(y, probs)
average_precision_score(y, probs)           # 0.4528
```

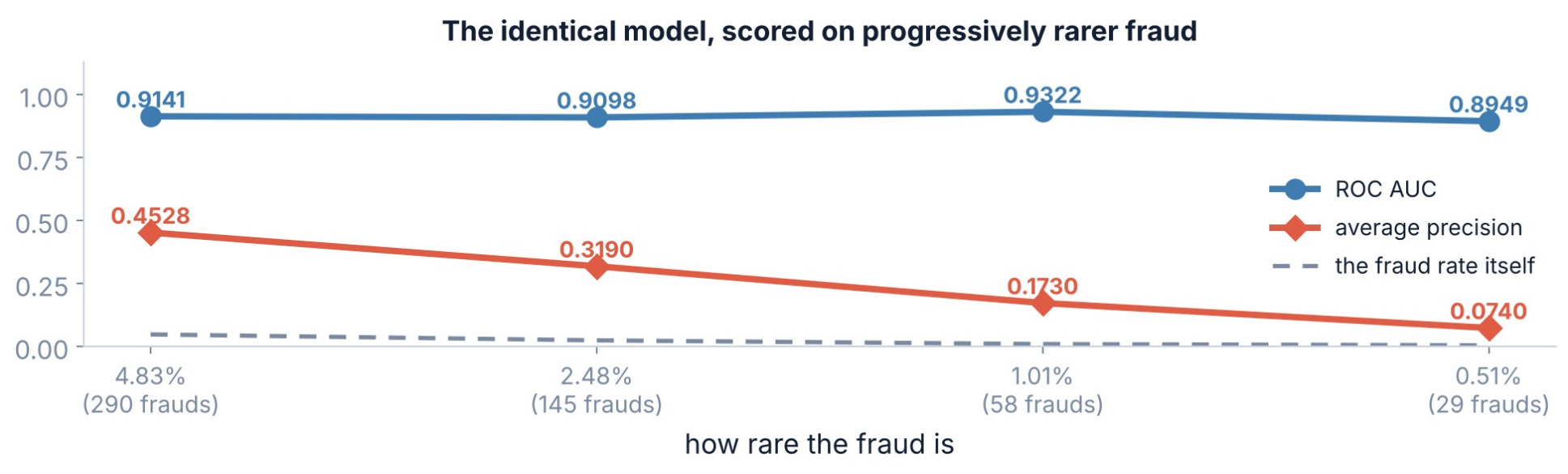
Every one of these takes probabilities, not predictions.

Pass .predict() output by mistake and the function still runs — it just draws a curve through a single point and reports a meaningless number. No error, no warning.

If an AUC ever comes back suspiciously round or low, check what you passed in first.

THE IMBALANCE TRAP

Where ROC quietly flatters you



The same model, every time. Only the share of fraud in the test set changed. ROC AUC held near 0.91; average precision fell from 0.4528 to 0.0740.

THE DENOMINATOR AGAIN

Why that happens, in one line of arithmetic

ROC's false positive rate divides by the number of honest payments — and on imbalanced data that number is enormous.

at this cut-off	false alarms	false positive rate	precision
0.05	970	0.1699 — looks small	0.2023 — four blocks in five are wrong
0.10	562	0.0984 — looks tiny	0.2739
0.20	216	0.0378 — looks excellent	0.4146

970 wrongly blocked cards is 17% of honest payments and sounds forgivable. It is also four out of every five blocks the bank makes, and that is what the customer feels.

A RULE YOU CAN KEEP

So which curve

USE ROC / AUC

comparing models on the same data

the classes are roughly balanced

you want one number that does not depend on the cut-off

USE PRECISION-RECALL / AP

one class is rare — under about 10%

you care about the positives specifically

you want a curve that notices when most of your alarms are false

USE NEITHER

you already know what each mistake costs

then expected cost beats both, and section 5 shows it

A useful habit: whenever somebody shows you an AUC above 0.9 on a rare event, ask for the precision-recall curve too. It takes one line and it often changes the conversation.

SECTION 5 OF 5

Putting a price on it

The only way to actually settle the question

- cost in rupees
- the cut-off falls out
- and more than two classes

EXPECTED COST

Stop arguing about metrics. Count the money.

```
#  a fraud we miss costs the bank the value of the payment
#  a good card we block costs the bank  COST_OF_A_BLOCK

missed = amount[(y == 1) & (pred == 0)].sum()
blocked = ((pred == 1) & (y == 0)).sum()
total  = missed + COST_OF_A_BLOCK * blocked
```

What this needs from you

One number: what a wrongly blocked card costs. Support time, goodwill, the chance the customer moves bank.

Nobody enjoys estimating it. But picking a metric instead does not avoid the estimate — it just hides it, and makes it badly.

THE TWO ENDS, FOR SCALE

Block nothing: ₹44,478 of fraud succeeds.

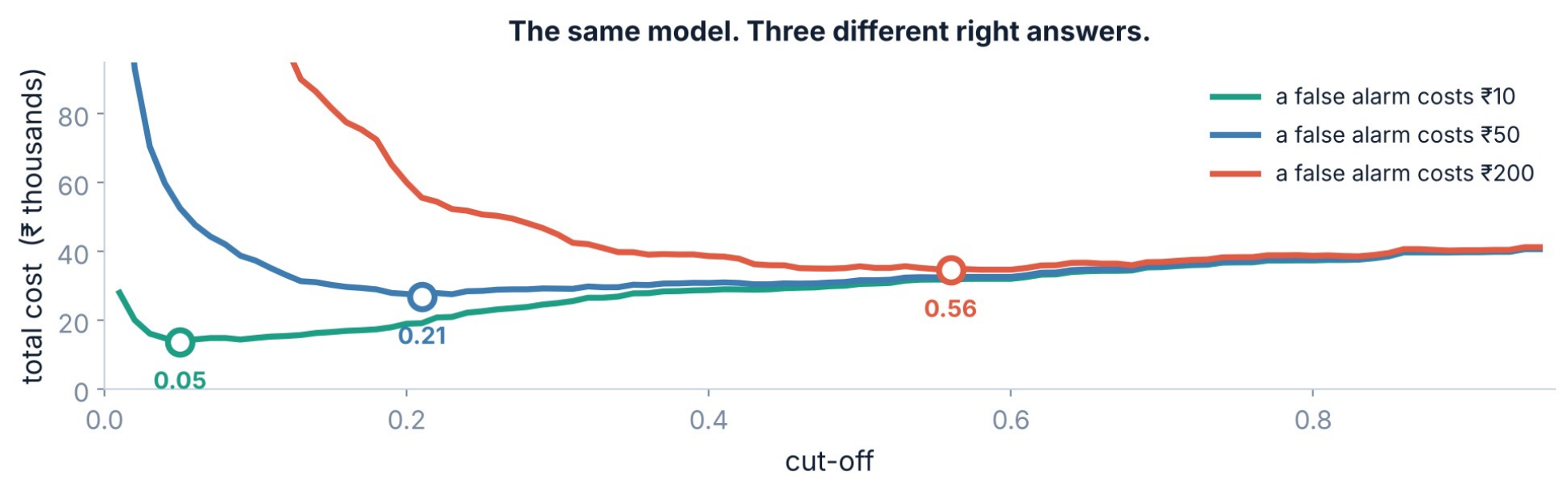
Block everything: no fraud, and 5,710 furious customers.

Everything sensible lives in between.

Note what this calculation does NOT need: a metric. It needs the confusion matrix and a price list.

THREE PRICE LISTS

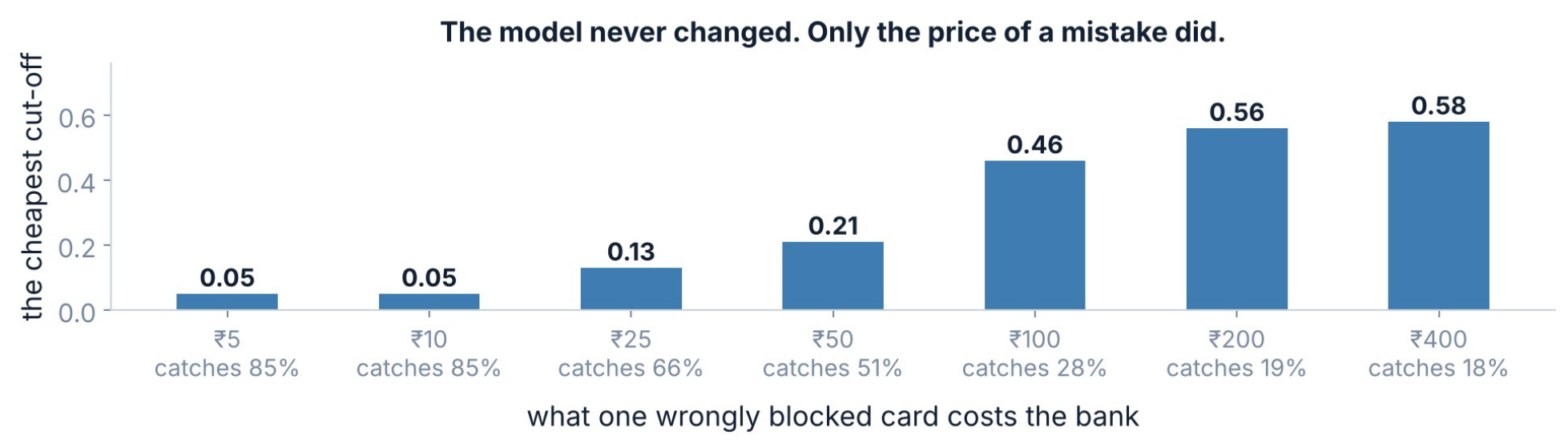
Total cost against the cut-off



Three curves, three minima: 0.05, 0.21 and 0.56. The model is identical in all three. Only what a blocked customer costs the bank is different.

NOT OF THE MODEL

The cut-off is a property of the business



At ₹5 a block, take the 0.05 cut-off and catch 85% of fraud. At ₹400, take 0.58 and catch 18%. Both are correct. The question was never 'which cut-off is best'.

THE MATRIX GROWS

More than two classes

1,800 support tickets, sorted into three priorities

actual \ predicted	low	medium	high
low	**1,063**	117	0
medium	221	**251**	23
high	9	65	**51**

accuracy 0.7583

low
F1 0.8597 recall 0.9008

medium
F1 0.5409 recall 0.5071

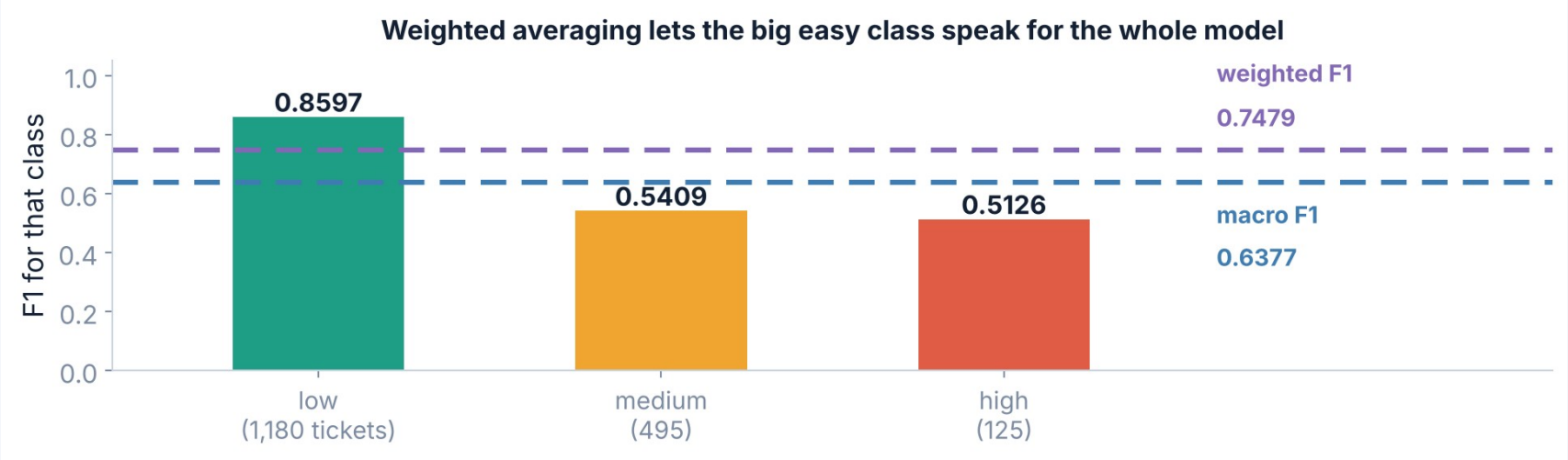
high
F1 0.5126 recall 0.4080

The 'high' row is the one that matters, and the model finds 51 of its 125 — while reporting 75.8% accuracy.

Precision and recall still work — they are just computed one class at a time, each class taking its turn as 'the positive one'.

MACRO, WEIGHTED, MICRO

Averaging across classes: the choice that hides things



average	what it does	here
macro	the plain mean of each class's F1 — every class counts the same	0.6377
weighted	weighted by how common each class is — big classes dominate	0.7479
micro	pools all the predictions first — equals accuracy for single-label problems	0.7583

Report macro when the rare class is the one you care about. 'High priority' is 7% of tickets, so weighted averaging lets the easy 'low' class answer for the whole model.

ONE CALL, EVERYTHING

classification_report, read properly

```
print(classification_report(y, pred, digits=4))

              precision    recall  f1-score   support

         low     0.8221    0.9008    0.8597      1180
      medium     0.5797    0.5071    0.5409       495
        high     0.6892    0.4080    0.5126       125

    accuracy                         0.7583      1800
   macro avg     0.6970    0.6053    0.6377      1800
weighted avg     0.7462    0.7583    0.7479      1800
```

Read it bottom-up

The three rows at the bottom are the summary everybody quotes.

The three rows at the top are where the problem actually is.

support is how many real cases of that class existed — always check it before trusting a rate computed from it.

'high' has recall 0.4080 on a support of 125. The model misses 74 of the 125 genuinely urgent tickets, and the accuracy line says 0.7583.

A REFERENCE YOU CAN KEEP

Which number to ask for

Which number to ask for

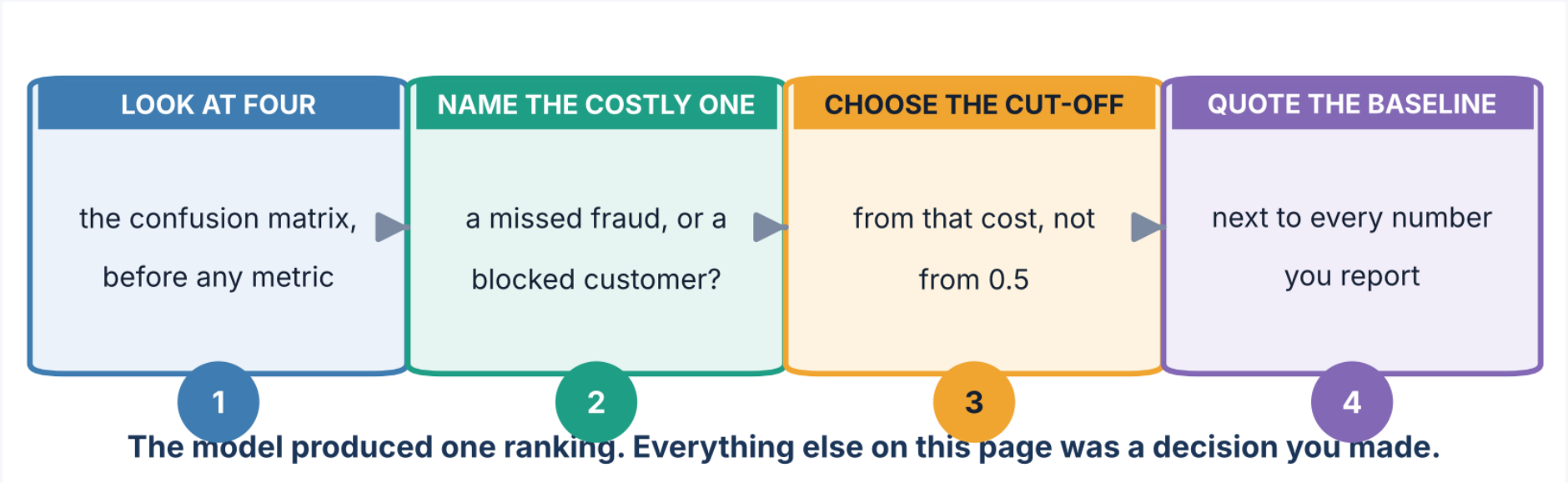
when the situation is…	ask for	because
balanced classes, mistakes cost the same	**accuracy**	nothing is hidden, and it is easy to explain
missing a case is the expensive mistake	**recall**	cancer screens, fraud, safety alerts
acting wrongly is the expensive mistake	**precision**	blocking a card, auto-rejecting an applicant
you need one number and both matter	**F1**	it will not let one hide the other
comparing models, not choosing a cut-off	**AUC or AP**	they score the ranking at every cut-off
one class is rare	**PR, not ROC**	ROC's false positive rate flatters you
you know what each mistake costs	**money**	no generic metric beats expected cost

Every row above is a question about the problem, not about the model.

Notice that the left-hand column never mentions the model. Every one of these is a question about the problem you are solving.

ALL OF TODAY ON ONE PAGE

The four habits



The model handed you one ranking of 6,000 payments. Everything after that — the cut-off, the metric, the number in the report — was a decision a person made.

A CHECKLIST

Mistakes this lecture should stop you making

if you catch yourself…	what is wrong	do this instead
reporting accuracy on rare events	'never block' scored 0.9517 here	quote the baseline beside it, always
quoting precision without recall	or recall without precision	quote both, or F1, and say which cut-off
using .predict() for a curve	that is one point, not a curve	pass .predict_proba()[:, 1]
showing an ROC curve for a rare class	the false positive rate stays flatteringly small	show the precision-recall curve too
choosing the cut-off that maximises accuracy	on imbalanced data that is close to doing nothing	choose it from what each mistake costs
reporting weighted F1 on an imbalanced problem	the big easy class answers for the whole model	report macro F1, or per-class

Five of these six make your headline number go up. That is what makes them survive review.

WHAT TO DO

Tonight's practice

A — Work it out

8 questions

Confusion matrices by hand, precision and recall from counts, reading a classification report

B — Find the mistake

4 questions

Four reports that quote a real number and mean something false

C — Write the code

5 tasks

Build the curves, choose a cut-off from cost, and defend it in writing

D — Challenge

2 tasks

Write AUC from scratch by counting pairs, then referee a disagreement between two teams

Part D asks you to compute AUC by sampling random fraud/legit pairs and counting which scored higher — and to land on roc_auc_score's answer.

WHAT TO REMEMBER

Five numbers from today

0.9583 — the accuracy of a fraud model that let 223 of 290 frauds through — against a 0.9517 baseline

0.7128 / 0.2310 — its precision and its recall at the same cut-off, pointing in opposite directions

0.039 — F1 for precision 0.95 with recall 0.02 — a plain average would have called it 0.485

0.9141 — ROC AUC, which barely moved as the fraud rate fell to 0.51% while average precision collapsed to 0.0740

0.05 → 0.58 — the cheapest cut-off as the price of a wrongly blocked card went from ₹5 to ₹400

Next: k-Nearest Neighbours — the first model that does not fit an equation at all.